

MINISTRY OF SCIENCE AND TECHNOLOGY
POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



Assignment 06

Lecturer	: Tran Dinh Que
Class	: E23TTTN02
Group	: 03
Student Name	: Lê Huy Hoàng
Student ID	: B23DCVT171

I. Task 1: Requirements Analysis

1. Actors

Customer – The main user who searches for products and orders and views returned results.

2. Functional Requirements

- Search products by keyword/text.
- Search products by voice.
- Search products by image.
- Search orders.
- View product information.
- View order information.

3. Non-Functional Requirements

- Usability: The system should be easy to use.
- Performance: Search results should be returned quickly.
- Maintainability: Components should be easy to modify and maintain.
- Extensibility: New search methods or AI models can be added.
- Modularity: UI, application logic, and data access should be separated.

4. Input Modalities

Mode	Input	Processing
Text	Keyword or natural-language query	Text search
Voice	Voice/audio query	Speech-to-text → text search
Image	Product image	Image similarity search

5. Search Outputs

- Product Search: A list of relevant products with product information.
- Order Search: Order information such as order ID, status, and total amount.

II. Task 2: Use Case Diagram

1. Actor: Customer: The main user who interacts with the system to search for products and orders and view related information.

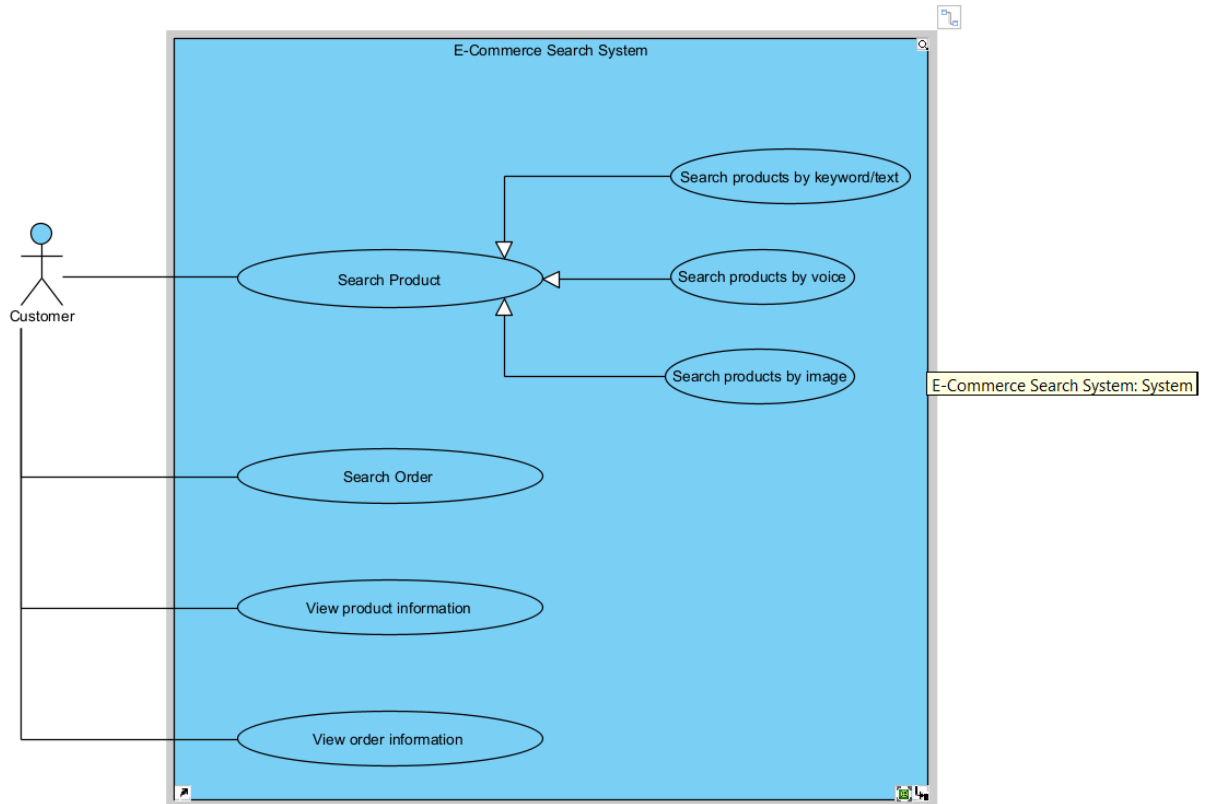
2. Main Use Cases

- Search Product: Allows the customer to search for products.
- Search products by keyword/text: Searches for products using keywords or natural-language text.
- Search products by voice: Searches for products using voice input.
- Search products by image: Searches for visually similar products using an image.
- Search Order: Allows the customer to search for order information.
- View product information: Allows the customer to view product details.
- View order information: Allows the customer to view order details.

3. Relationships

- The Customer has direct associations with:
 - Customer → Search Product

- Customer → Search Order
- Customer → View product information
- Customer → View order information
- Search Product is a general use case with three specialized search methods:
 - Search products by keyword/text → Search Product
 - Search products by voice → Search Product
 - Search products by image → Search Product



III. Task 3: Three-Layer Architecture

The system is organized into three layers: Presentation Layer, Application / Intelligence Layer, and Data Layer. Each layer has a separate responsibility, and the dependencies follow the direction:

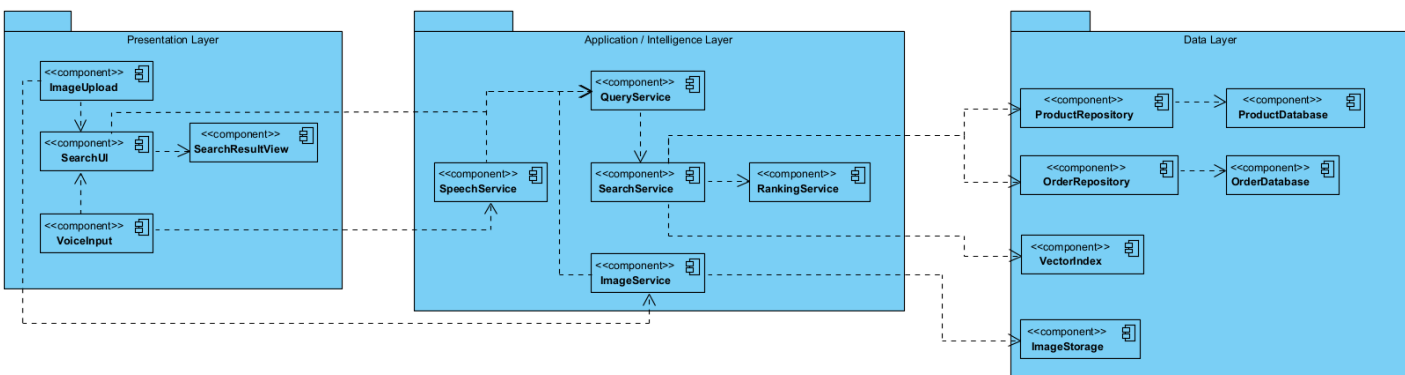
Presentation Layer → Application / Intelligence Layer → Data Layer.

1. **Presentation Layer**: contains SearchUI, VoicelInput, ImageUpload, and SearchResultView. This layer receives user input and displays the returned search results.
2. **Application / Intelligence Layer**: contains QueryService, SpeechService, ImageService, SearchService, and RankingService. This layer processes different input types, creates a common query representation, retrieves relevant data, and ranks the results.
3. **Data Layer**: contains ProductRepository, OrderRepository, VectorIndex, ProductDatabase, OrderDatabase, and ImageStorage. This layer stores and provides access to product, order, image, and vector data.

4. Main Interaction Flows

- **Text Search Flow**

- SearchUI → QueryService → SearchService → ProductRepository → ProductDatabase
- The user enters a text query through SearchUI. QueryService creates the query representation, and SearchService retrieves matching products from ProductRepository.
- **Voice Search Flow**
 - VoiceInput → SpeechService → QueryService → SearchService → ProductRepository
 - VoiceInput receives the user's voice input. SpeechService converts it into text, then QueryService creates the query representation and sends it to SearchService. This follows the voice-search process described in the assignment.
- **Image Search Flow**
 - ImageUpload → ImageService → QueryService → SearchService → VectorIndex
 - The user uploads an image through ImageUpload. ImageService processes the image, while QueryService creates the image-query representation. SearchService then uses VectorIndex to retrieve similar products. ImageService can also access ImageStorage for image data.
- **Order Search Flow**
 - SearchUI → QueryService → SearchService → OrderRepository → OrderDatabase
 - The system processes the order query and retrieves order information through OrderRepository.
- **Ranking Flow**
 - SearchService → RankingService
 - After relevant candidates are retrieved, RankingService ranks them before the results are presented to the user. The assignment explicitly separates retrieval from ranking.



IV. Task 4: Sequence Diagram

The Sequence Diagram describes the interaction when a Customer searches for a product using voice input. The diagram contains the required participants: Customer, SearchUI, SpeechService, QueryService, SearchService, ProductRepository, and RankingService.

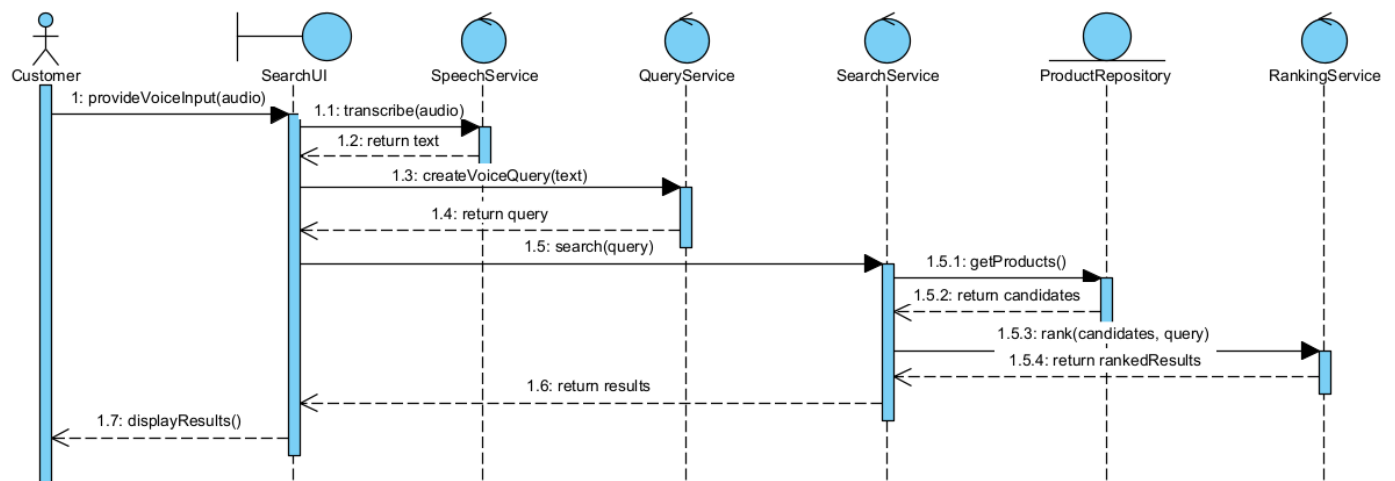
1. Participants

Participant	Responsibility
-------------	----------------

Customer	Provides the voice query and receives the results.
SearchUI	Receives user input and displays search results.
SpeechService	Converts voice input into text.
QueryService	Creates a query representation from the transcribed text.
SearchService	Retrieves relevant products.
ProductRepository	Provides product data for the search process.
RankingService	Ranks the retrieved product candidates.

2. Interaction Flow

- 1. The Customer provides voice input to SearchUI.
- 2. SearchUI sends the audio input to SpeechService.
- 3. SpeechService converts the voice input into text and returns it.
- 4. SearchUI sends the text to QueryService to create a voice query representation.
- 5. QueryService returns the generated query.
- 6. SearchUI sends the query to SearchService.
- 7. SearchService retrieves products from ProductRepository.
- 8. ProductRepository returns the candidate products.
- 9. SearchService sends the candidates to RankingService.
- 10. RankingService ranks the candidates and returns the ranked results.
- 11. SearchService returns the results to SearchUI.
- 12. SearchUI displays the results to the Customer.



V. Task 5: Python Implementation

The Python prototype implements the five required functions using three layers: presentation for input and output, application for search and ranking, and data for storage. `main.py` creates the repositories and services and connects them to `SearchUI`.

`main.py` connects the data repositories and application services to `SearchUI`. The listings below contain the relevant project methods; imports and other helper methods remain in their source files.

CodePython/main.py — build_ui

```
def build_ui():
    """Tạo kho dữ liệu và các service, sau đó truyền chúng vào SearchUI."""
    repository = ProductRepository()
    vector_index = VectorIndex(repository.all_products())
    # Console và web dùng chung các thành phần này để thống nhất cách tìm kiếm.
    return SearchUI(
        QueryService(),
        SpeechService(),
        SearchService(repository, vector_index, ImageService()),
        RankingService(),
    )
```

1 Product repository

- `ProductRepository` locates `products.json` beside its source file and loads the 20 product records.
- It validates each record and rejects duplicate IDs before building an ID lookup dictionary.
- Search services receive independent copies through `all_products()` or `find_by_id()`; data remains in local JSON.

CodePython/data/product_repository.py — ProductRepository: __init__, _validate_product, all_products, find_by_id

```
def __init__(self, data_path=None):
    """Đọc products.json mặc định hoặc file được chỉ định và kiểm tra dữ liệu."""
    # Lấy đường dẫn theo vị trí file Python để chạy từ folder nào cũng được.
    self.data_path = (
        Path(data_path) if data_path is not None
        else Path(__file__).resolve().with_name("products.json")
    )
    try:
        with self.data_path.open(encoding="utf-8") as source:
            products = json.load(source)
    except FileNotFoundError as exc:
        raise FileNotFoundError(
            f"Product data file not found: {self.data_path}"
        ) from exc
    except (json.JSONDecodeError, UnicodeDecodeError) as exc:
        raise ValueError(f"Invalid product JSON: {self.data_path}: {exc}") from exc
    if not isinstance(products, list):
        raise ValueError("Product data must be a JSON list.")
    # Kiểm tra từng bản ghi và cấm ID trùng trước khi lập bảng tra cứu.
    seen = set()
```

```

for position, product in enumerate(products, start=1):
    self._validate_product(product, position)
    product_id = product["id"]
    if product_id in seen:
        raise ValueError(f"Duplicate product id: {product_id}")
    seen.add(product_id)
self._products = products
# Dictionary giúp tra theo ID mà không phải duyệt toàn bộ danh sách.
self._by_id = {product["id"]: product for product in products}

@staticmethod
def _validate_product(product, position):
    """Kiểm tra ID, tên/danh mục/màu, giá, tồn kho và từ khóa bổ sung."""
    prefix = f"Product record {position}"
    if not isinstance(product, dict):
        raise ValueError(f"{prefix} must be an object.")
    product_id = product.get("id")
    if not isinstance(product_id, int) or isinstance(product_id, bool):
        raise ValueError(f"{prefix} must have an integer id.")
    for field in ("name", "category", "color"):
        value = product.get(field)
        if not isinstance(value, str) or not value.strip():
            raise ValueError(f"{prefix}: {field} must be a nonempty string.")
    # Giá phải hữu hạn và không âm; bool không được coi là số giá hợp lệ.
    price = product.get("price")
    try:
        valid_price = (
            isinstance(price, (int, float)) and not isinstance(price, bool)
            and math.isfinite(price) and price >= 0
        )
    except OverflowError:
        valid_price = False
    if not valid_price:
        raise ValueError(f"{prefix}: price must be a finite nonnegative number.")
    stock = product.get("stock")
    if not isinstance(stock, int) or isinstance(stock, bool) or stock < 0:
        raise ValueError(f"{prefix}: stock must be a nonnegative integer.")
    terms = product.get('search_terms', [])
    if not isinstance(terms, list) or any(not isinstance(term, str) for term in
terms):
        raise ValueError(f"{prefix}: search_terms must be a list of strings.")

def all_products(self):
    """Trả bản sao sâu để nơi gọi không sửa nhầm dữ liệu gốc trong kho."""
    return deepcopy(self._products)

def find_by_id(self, product_id):
    """Trả bản sao sản phẩm theo ID hoặc None khi ID không có trong kho."""
    product = self._by_id.get(product_id)
    return deepcopy(product) if product is not None else None

```

2 Text search

- SearchUI receives the text; QueryService trims it and creates a query with type text.
- SearchService lowercases the query, maps known Vietnamese category phrases, and matches words against names, categories, colors, and search terms.
- Each matching word adds one point. Nonmatching products are excluded; the shared ranking and display flow handles the candidates.

CodePython/application/query_service.py — QueryService: _normalize_text, text_query

```
@staticmethod
def _normalize_text(text):
    """Kiểm tra kiểu chuỗi và bỏ khoảng trắng thừa ở hai đầu."""
    if not isinstance(text, str):
        raise ValueError("Text and voice queries must be strings.")
    return text.strip()

def text_query(self, text):
    """Gắn loại text cho nội dung tìm kiếm đã được kiểm tra."""
    return {"type": "text", "query": self._normalize_text(text)}
```

CodePython/application/search_service.py — SearchService: search_text

```
def search_text(self, query_text):
    """Mỗi từ truy vấn khớp chuỗi thông tin sản phẩm đóng góp một điểm."""
    if not isinstance(query_text, str):
        raise ValueError("Text search query must be a string.")
    normalized = query_text.lower()
    # Đổi cụm danh mục tiếng Việt sang tiếng Anh trước khi tách từ.
    # Dùng ranh giới từ để "bàn phím" không bị khớp nhầm với "bang".
    aliases = {'điện thoại': 'phone', 'dien thoai': 'phone',
               'máy tính bảng': 'tablet', 'may tinh bang': 'tablet',
               'máy tính xách tay': 'laptop', 'may tinh xach tay': 'laptop',
               'bàn phím': 'keyboard', 'ban phim': 'keyboard',
               'chuột': 'mouse', 'chuot': 'mouse',
               'phụ kiện': 'accessory', 'phu kien': 'accessory'}
    for phrase, category in aliases.items():
        normalized = re.sub(r'(?<!w)' + re.escape(phrase) + r'(?!\w)', category,
normalized)
        words = normalized.split()
        if not words:
            return []
        candidates = []
        # Tìm trong tên, danh mục, màu và từ khóa bổ sung; đây là khớp chuỗi con,
        # chưa phải tìm kiếm ngữ nghĩa hay phân tích điều kiện giá trong câu.
        for product in self.repository.all_products():
            searchable = " ".join(
                product[field] for field in ("name", "category", "color")
            ).lower() + " " + " ".join(product.get("search_terms", [])).lower()
            score = sum(word in searchable for word in words)
            # Chỉ trả sản phẩm khớp ít nhất một từ; chưa sắp xếp ở bước này.
            if score > 0:
                candidates.append((product, float(score)))
    return candidates
```

CodePython/presentation/search_ui.py — SearchUI: search_text

```
def search_text(self, text, top_k=None):
    """Tạo truy vấn text và báo lỗi đầu vào thân thiện thay vì in traceback."""
    print("\n=== TEXT SEARCH ===")
    print("Processing mode: Text")
    print("Input:", text)
    try:
        query = self.query_service.text_query(text)
        return self._retrieve_rank_display(query, top_k=top_k)
    except (ValueError, TypeError) as exc:
        print(f"Validation error: {exc}")
    return []
```

3 Simulated voice search

- SearchUI accepts an already-transcribed string, such as "find laptop"; no audio is recorded.
- SpeechService checks that the input is text and returns it unchanged; QueryService creates a voice query.
- SearchService processes voice queries through the text-search algorithm, then the shared flow ranks and displays the products.

CodePython/application/speech_service.py — SpeechService: transcribe

```
def transcribe(self, audio_input):
    """Kiểm tra đầu vào rồi trả nguyên văn bản phiên âm, không nhận dạng âm thanh."""
    if not isinstance(audio_input, str):
        raise ValueError("Simulated voice input must be already-transcribed text.")
    return audio_input
```

CodePython/application/query_service.py — QueryService: voice_query

```
def voice_query(self, text):
    """Gắn loại voice cho văn bản phiên âm do SpeechService trả về."""
    return {"type": "voice", "query": self._normalize_text(text)}
```

CodePython/presentation/search_ui.py — SearchUI: search_voice

```
def search_voice(self, audio_input, top_k=None):
    """Mô phỏng phiên âm, tạo truy vấn voice rồi chạy luồng tìm kiếm chung."""
    print("\n=== VOICE SEARCH ===")
    print("Processing mode: Voice (simulated speech-to-text)")
    print("Voice input:", audio_input)
    try:
        text = self.speech_service.transcribe(audio_input)
        print("Transcribed text:", text)
        query = self.query_service.voice_query(text)
        return self._retrieve_rank_display(query, top_k=top_k)
    except (ValueError, TypeError) as exc:
        print(f"Validation error: {exc}")
    return []
```

4 Image similarity search

- QueryService validates a manually supplied vector. VectorIndex provides the artificial product vectors stored in products.json.
- SearchService checks the dimension and retrieves each indexed product; ImageService calculates cosine similarity for each vector pair.
- The shared flow ranks the similarity scores and displays the products. A zero vector scores 0; no photo features are extracted.

CodePython/application/query_service.py — QueryService: image_query

```
def image_query(self, embedding):
    """Kiểm tra vector số trước khi tạo truy vấn loại image."""
    return {"type": "image", "embedding": normalize_embedding(embedding)}
```

CodePython/application/image_service.py — ImageService: cosine_similarity

```
def cosine_similarity(self, a, b):
    """Tính cosine trong khoảng [-1, 1]; vector toàn số 0 trả điểm 0."""
    left = np.asarray(normalize_embedding(a), dtype=np.float64)
    right = np.asarray(normalize_embedding(b), dtype=np.float64)
    if left.size != right.size:
        raise ValueError(
            f"Embedding dimension mismatch: {left.size} and {right.size}."
        )
    # Chia theo phần tử lớn nhất trước để tránh tràn hoặc hụt số
    # khi tính tích vô hướng và độ dài với các giá trị quá lớn/nhỏ.
    left_scale = float(np.max(np.abs(left)))
    right_scale = float(np.max(np.abs(right)))
    if left_scale == 0.0 or right_scale == 0.0:
        return 0.0
    left /= left_scale
    right /= right_scale
    # Đưa mỗi vector về độ dài 1, sau đó lấy tích vô hướng làm điểm cosine.
    left /= np.linalg.norm(left)
    right /= np.linalg.norm(right)
    # Giới hạn kết quả để sai số số thực không vượt ra ngoài [-1, 1].
    return float(np.clip(np.dot(left, right), -1.0, 1.0))
```

CodePython/application/search_service.py — SearchService: search_image

```
def search_image(self, query_embedding):
    """So sánh vector truy vấn với các vector sản phẩm có cùng số chiều."""
    vector = normalize_embedding(query_embedding)
    dimension = self.vector_index.dimension
    if dimension is not None and len(vector) != dimension:
        raise ValueError(
            f"Image query dimension {len(vector)} does not match "
            f"product embedding dimension {dimension}."
        )
    candidates = []
    # Chỉ mục lưu ID và vector; lấy thông tin sản phẩm tương ứng từ repository.
    for product_id, embedding in self.vector_index.all_embeddings().items():
        product = self.repository.find_by_id(product_id)
        if product is None:
            continue
```

```

        score = self.image_service.cosine_similarity(vector, embedding)
        candidates.append((product, float(score)))
    return candidates

```

CodePython/presentation/search_ui.py — SearchUI: search_image

```

def search_image(self, embedding, top_k=None):
    """Tìm bằng vector nhân tạo và hiển thị điểm với nhãn similarity."""
    print("\n=== IMAGE SEARCH ===")
    print("Processing mode: Image (artificial embedding; cosine similarity)")
    print("Query embedding:", embedding)
    try:
        query = self.query_service.image_query(embedding)
        return self._retrieve_rank_display(query, top_k=top_k,
score_label="similarity")
    except (ValueError, TypeError) as exc:
        print(f"Validation error: {exc}")
        return []

```

5 Result ranking and display

- SearchService returns (product, score) candidates; RankingService checks their structure and finite scores.
- Candidates are sorted by score in descending order. Equal scores retain their original order, and top_k limits the ranked list.
- SearchUI displays the product details and scores through a shared retrieval, ranking, and display method.

CodePython/application/ranking_service.py — RankingService: rank

```

def rank(self, candidates, top_k=None):
    """Kiểm tra các cặp (sản phẩm, điểm) rồi lấy tối đa top_k kết quả tốt nhất."""
    if top_k is not None and (
        not isinstance(top_k, int) or isinstance(top_k, bool) or top_k < 0
    ):
        raise ValueError("top_k must be a nonnegative integer or None.")
    try:
        entries = list(candidates)
    except TypeError as exc:
        raise ValueError("Candidates must be an iterable of (product, score)
pairs.") from exc
    # Không cho phép điểm NaN/vô cực hoặc dữ liệu sai cấu trúc tham gia xếp hạng.
    for candidate in entries:
        if not isinstance(candidate, (tuple, list)) or len(candidate) != 2:
            raise ValueError("Every candidate must be a (product, score) pair.")
        score = candidate[1]
        try:
            valid_score = (
                isinstance(score, Real) and not isinstance(score, bool)
                and math.isfinite(score)
            )
        except OverflowError:
            valid_score = False
        if not valid_score:
            raise ValueError("Candidate scores must be finite numbers.")

```

```
# sorted giữ thứ tự ban đầu của các phần tử đồng điểm (sắp xếp ổn định).
ranked = sorted(entries, key=lambda candidate: candidate[1], reverse=True)
return ranked if top_k is None else ranked[:top_k]
```

CodePython/presentation/search_ui.py — SearchUI: _retrieve_rank_display, display_results

```
def _retrieve_rank_display(self, query, top_k=None, score_label="score"):
    """Luồng dùng chung: tìm ứng viên → xếp hạng → hiển thị kết quả."""
    candidates = self.search_service.search(query)
    results = self.ranking_service.rank(candidates, top_k=top_k)
    self.display_results(results, score_label=score_label)
    return results

def display_results(self, results, top_k=None, score_label="score"):
    """In thông tin sản phẩm và điểm, hỗ trợ giới hạn số kết quả hiển thị."""
    if not results:
        print("No products found.")
        return

    displayed = results if top_k is None else results[:top_k]
    for position, (product, score) in enumerate(displayed, start=1):
        print(f"{position}. {product['name']}")
        print(
            f"    category: {product['category']} | color: {product['color']}"
            f" | price: {product['price']} | stock: {product['stock']}"
            f" | {score_label}: {score:.4f}"
        )
```

Implementation evidence

```
def build_ui():
    """Create storage and services, then inject them into the presentation layer."""
    repository = ProductRepository()
    vector_index = VectorIndex(repository.all_products())
    return SearchUI(
        QueryService(),
        SpeechService(),
        SearchService(repository, vector_index, ImageService()),
        RankingService(),
    )
```

```
PS C:\In Class\Ky 1 nam 4\Thiết kế hệ thống thông tin\Baitap\KiemTra3Tiet> python .\CodePython\main.py --top-k 3
=== E-Commerce Search Demo ===
```

```
=== TEXT SEARCH ===
```

```
Processing mode: Text
```

```
Input: phone
```

1. Apple iPhone 16
category: phone | color: ultramarine | price: 799 | stock: 12 | score: 1.0000
2. Apple iPhone 16 Plus
category: phone | color: teal | price: 899 | stock: 9 | score: 1.0000
3. Apple iPhone 16 Pro
category: phone | color: desert titanium | price: 999 | stock: 7 | score: 1.0000

```
=== VOICE SEARCH ===
```

```
Processing mode: Voice (simulated speech-to-text)
```

```
Voice input: find laptop
```

```
Transcribed text: find laptop
```

1. Apple MacBook Air 13-inch M4 (2025)
category: laptop | color: sky blue | price: 1049 | stock: 8 | score: 1.0000
2. Apple MacBook Air 15-inch M4 (2025)
category: laptop | color: midnight | price: 1249 | stock: 5 | score: 1.0000
3. Apple MacBook Pro 14-inch M4 (2024)
category: laptop | color: space black | price: 1649 | stock: 6 | score: 1.0000

```
=== IMAGE SEARCH ===
```

```
Processing mode: Image (artificial embedding; cosine similarity)
```

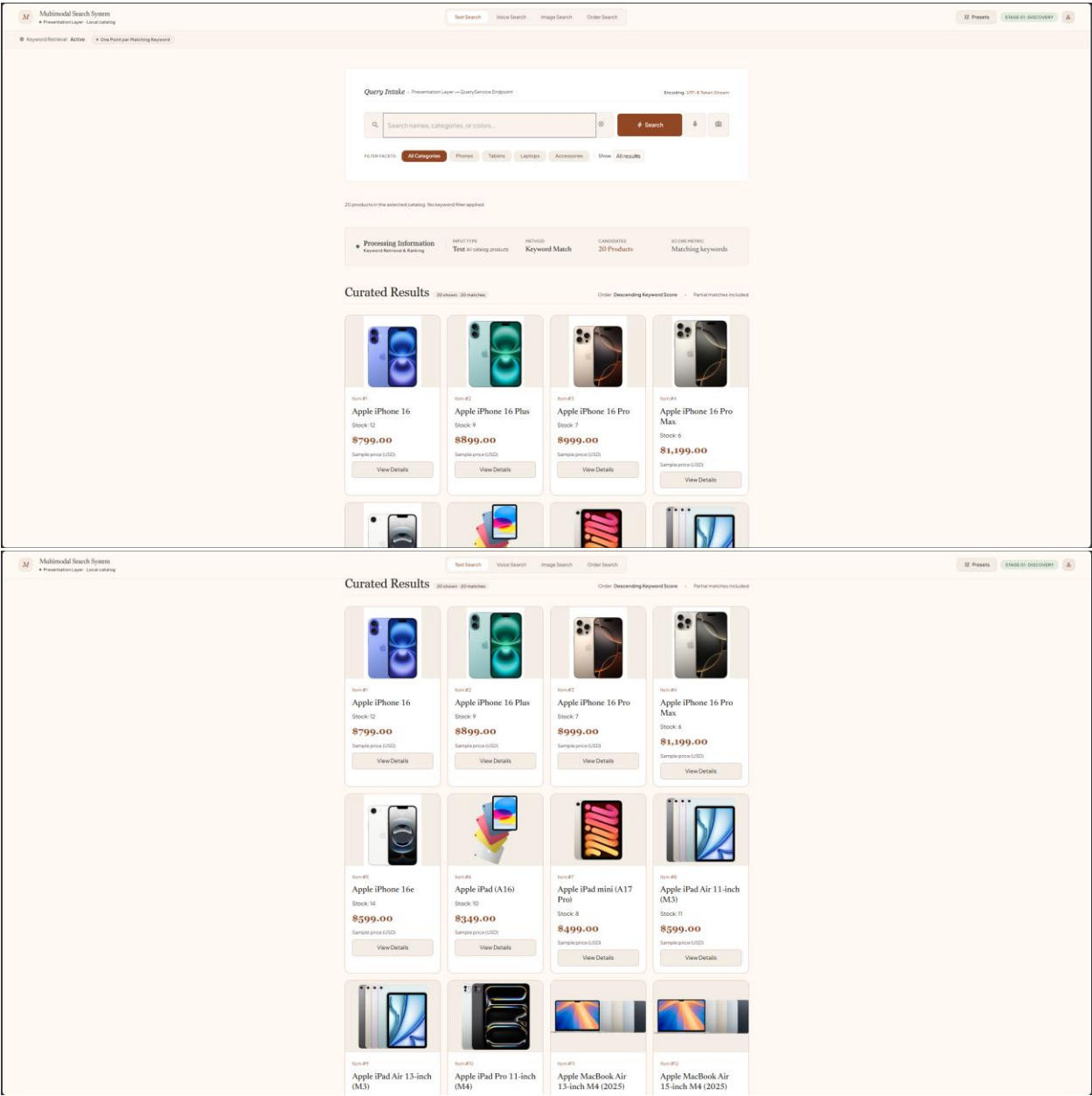
```
Query embedding: [0.95, 0.1, 0.05]
```

1. Apple iPhone 16
category: phone | color: ultramarine | price: 799 | stock: 12 | similarity: 1.0000
2. Apple iPhone 16 Pro
category: phone | color: desert titanium | price: 999 | stock: 7 | similarity: 0.9995
3. Apple iPhone 16 Plus
category: phone | color: teal | price: 899 | stock: 9 | similarity: 0.9992

VI. Task 6: Demonstration

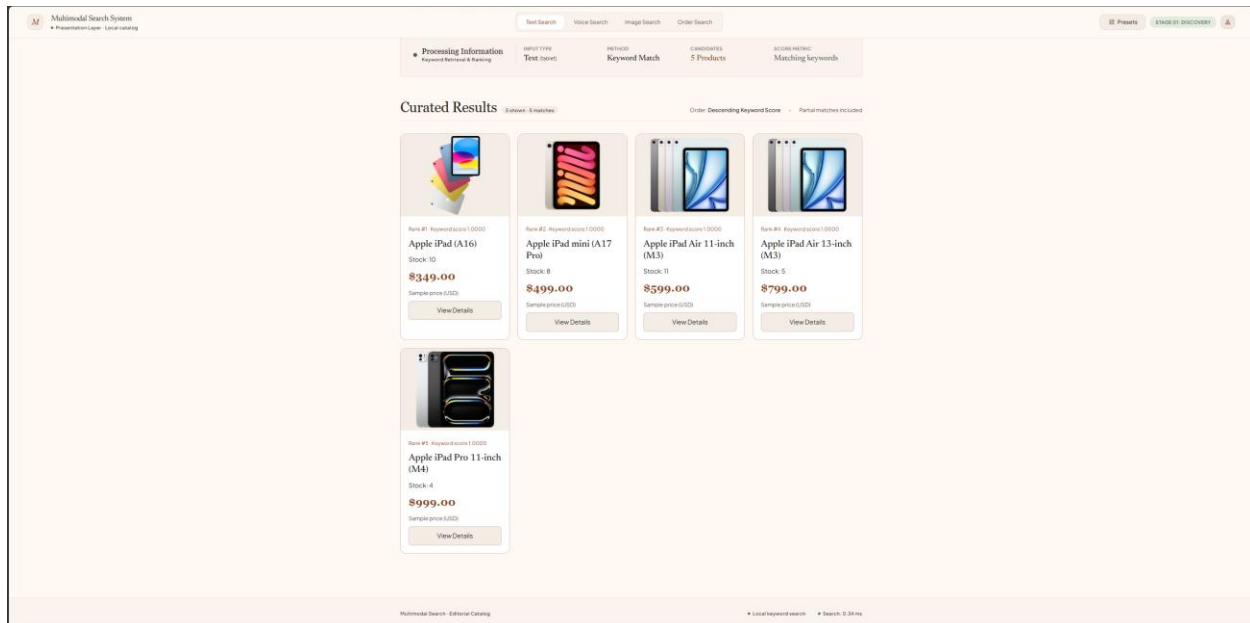
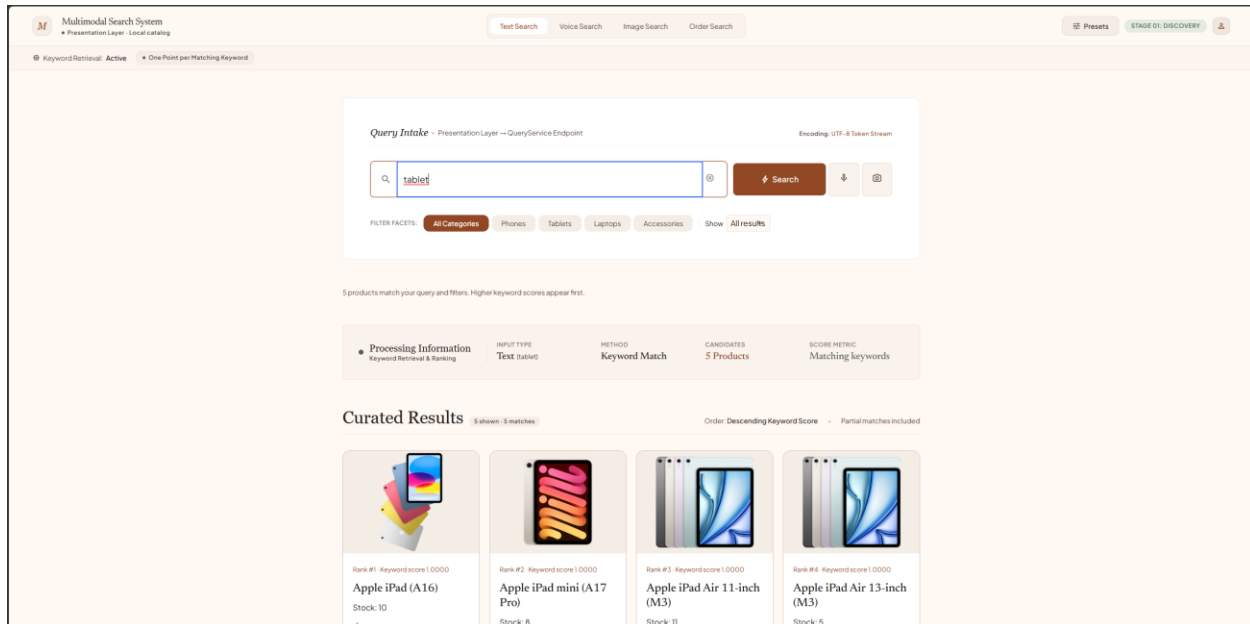
1. Home

The home page displays all 20 products across phones, tablets, laptops, and accessories when no keyword is entered. Users can select a category or open product details. This is catalog browsing, so item numbers are shown instead of relevance scores.



2. Text search

Input: "tablet" with All Categories selected. The system lowercases the query and matches its words against product names, categories, colors, and search terms. Five tablets are returned: iPad (A16), iPad mini (A17 Pro), iPad Air 11-inch (M3), iPad Air 13-inch (M3), and iPad Pro 11-inch (M4). Each has a keyword score of 1.0000; ties retain catalog order.



3. Voice Search

Input: "find phone" entered as a simulated transcript. No microphone or real speech recognition is used; the transcript is passed to the text-search service. Five phones are returned, led by iPhone 16, iPhone 16 Plus, and iPhone 16 Pro. Each scores 1.0000 because "phone" matches the product category; "find" adds no points.

The screenshot displays the 'E-Commerce Multimodal Search' interface. At the top, there are tabs for 'Text Search', 'Voice Search' (selected), 'Image Search', and 'Order Search'. Below the tabs, a progress bar shows steps 1 through 6. The main content area is divided into two sections. The left section, titled 'VOICE SIMULATION STUDIO', contains a 'Your Spoken Query' input field with the text 'find phone'. Below this is a waveform visualization and a 'Simulate Voice Search' button. The right section, titled 'PIPELINE SUMMARY Execution Telemetry', shows the input mode as 'Voice (Simulated Transcript)' and the transcript text as 'find phone'. It also displays 'CANDIDATES RETRIEVED' as 'Simulated: 5 after filters' and 'PROCESSING PIPELINE' as 'Simulated STT -> Keyword Search -> Ranking'. The bottom section, titled 'RANKED SELECTIONS', shows '5 Ranked Results' ordered by 'Keyword Match Score'. The results are as follows:

Rank	Product Name	Stock	Price	Sample price (USD)
Rank #1	Apple iPhone 16	Stock: 12	\$799.00	Sample price (USD)
Rank #2	Apple iPhone 16 Plus	Stock: 9	\$899.00	Sample price (USD)
Rank #3	Apple iPhone 16 Pro	Stock: 7	\$999.00	Sample price (USD)
Rank #4				
Rank #5				

The next screenshot shows a separate laptop-search example: five laptops are returned, led by MacBook Air 13-inch (M4, 2025). All five have a keyword score of 1.0000.

The screenshot displays the 'E-Commerce Multimodal Search' interface for a laptop search. The 'Voice Search' tab is selected, and the 'Your Spoken Query' input field contains the text 'find laptop'. The 'PIPELINE SUMMARY Execution Telemetry' section shows the input mode as 'Voice (Simulated Transcript)' and the transcript text as 'find laptop'. The 'CANDIDATES RETRIEVED' is 'Simulated: 5 after filters', and the 'PROCESSING PIPELINE' is 'Simulated STT -> Keyword Search -> Ranking'. The 'RANKED SELECTIONS' section shows '5 Ranked Results' ordered by 'Keyword Match Score'. The results are as follows:

Rank	Product Name	Stock	Price	Sample price (USD)
Rank #1	Apple MacBook Air 13-inch M4 (2025)	Stock: 8	\$1,049.00	Sample price (USD)
Rank #2	Apple MacBook Air 15-inch M4 (2025)	Stock: 5	\$1,249.00	Sample price (USD)
Rank #3	Apple MacBook Pro 14-inch M4 (2024)	Stock: 6	\$1,649.00	Sample price (USD)
Rank #4	Apple MacBook Pro 16-inch M4 Pro (2024)	Stock: 3	\$2,549.00	Sample price (USD)
Rank #5	ASUS Zenbook 14 OLED UX3405	Stock: 0	\$1,199.00	Sample price (USD)

4. Image search

Input: the iPhone 16 preset with vector [0.95, 0.1, 0.05]. The system compares this vector with all 20 product vectors using cosine similarity and ranks results in descending order. iPhone 16 ranks first (1.0000), followed by iPhone 16 Pro (0.9995) and iPhone 16 Plus (0.9992). These manually assigned three-dimensional vectors simulate image search; they are not extracted from the photos.

E-Commerce Multimodal Search System

Local Multimodal Catalogue

Text SearchVoice SearchImage SearchOrder Search

00 PresetsSEARCH BY DOCUMENTA

Artificial vectors. NumPy cosine similarity. 3-dimensions

CURATED REFERENCE QUERIES

Phones: Apple iPhone 16Tablets: Apple iPad (A16)Laptops: Apple MacBook Air (13-inch M4 (2025))Accessories: Logitech MX Master 3S Bluetooth Edition Mouse

Artificial Vectors

QUERY REFERENCE PREVIEW



Apple iPhone 16 - real product image

VECTOR VALUE DATA

Local Image Preview

Drag (PNG or JPEG) - maximum 12 MB

Browse File

Real manufacturer images accompany predefined artificial vectors. A custom image stays in this browser; no image is uploaded and no features are extracted. Enter its three vector values manually before searching.

MANUAL QUERY VECTOR: 3-FINITE NUMBERS

RETRIEVAL & RANKING

Processing Overview

Service duration: 7.77 ms

INPUT MODALITY

Image query vector

[0.95, 0.1, 0.05]

METHOD APPLIED

Cosine Similarity

Artificial 3-dimensional product embeddings

FILTERED CANDIDATES

20 of 20 candidates

Maximum similarity threshold

RANKING STRATEGY

Cosine Metric 1

$\text{vec}(Q) \cdot \text{vec}(P) = (A \cdot B) / (|A| \cdot |B|)$

Editorial Note on This Prototype

Search compares the input query vector with the local catalogue's artificial vectors. Similarity is a mathematical score between -1 and 1, not a recognition confidence or a measured visual attribute.

Index in-memory product vectors: Precision: Float64

Best Match: 1.0000

E-Commerce Multimodal Search System

Local Multimodal Catalogue

Text SearchVoice SearchImage SearchOrder Search

00 PresetsSEARCH BY DOCUMENTA

Search Similar Products

Returned 20 products ranked by cosine similarity

VECTOR MATCH SORTING: DESCENDING

Ranked Results via Artificial Embeddings

20 ranked results



Rank #1: similarity: 1.0000

Apple iPhone 16

Stock: 12

\$799.00

Sample price (USD)

View Details



Rank #2: similarity: 0.9995

Apple iPhone 16 Pro

Stock: 7

\$899.00

Sample price (USD)

View Details



Rank #3: similarity: 0.9992

Apple iPhone 16 Plus

Stock: 9

\$899.00

Sample price (USD)

View Details



Rank #4: similarity: 0.9990

Apple iPhone 16 Pro Max

Stock: 6

\$1,199.00

Sample price (USD)

View Details



Rank #5: similarity: 0.7967

Apple iPhone 16e

Stock: 14

\$599.00

Sample price (USD)

View Details



Rank #6: similarity: 0.7583

Apple iPad Air 11-inch (M3)

Stock: 11

\$599.00

Sample price (USD)

View Details



Rank #7: similarity: 0.7507

Apple iPad (A16)

Stock: 10

\$349.00

Sample price (USD)

View Details



Rank #8: similarity: 0.7292

Apple iPad mini (A17 Pro)

Stock: 8

\$499.00

Sample price (USD)

View Details

5. Order

Input: order ID "0001". An exact ID lookup returns a Shipped sample order containing one iPhone 16 and two Logitech MX Master 3S mice. The subtotal is \$989.00 and shipping is \$5.00, giving a total of \$994.00. This read-only order lookup does not use relevance scores or process a real payment.

The Archive & Order Ledger

MULTI-TENANTIAL COMMERCE DESIGN

FULFILLMENT ARCHIVE / Local sample order ledger

Lookup duration: 0.20 ms

Text Search

Voice Search

Image Search

Order Search

Saved Products

AMT

ORDER RECORD & DISPATCH

Order Lookup & Fulfillment Registry

Look up the local sample ledger to review customer details, record past fulfillment events and itemized totals.

Order ID SearchCustomer HistoryReturn Log

SEARCH BY ORDER NUMBER

OOO1

Retrieve OrderReset

Sample IDs: OOO1 - OOO2 - OOO3

Retrieved order #OOO1 Shipped

SUCCESS

SELECTED ORDERNet #OOO1

ORDER RECORD

Order #OOO1Shipped

CUSTOMERORDER DATE

Sample Customer2026-10-01

A

ID: OOO1

Recorded events completed2/3

ORDER TOTALMANIFEST

\$999.002 product lines3 units

Print Order Summary

ORDER DISCOVERY NOTE

These three sample records are read from the Python order repository. History, shared invoices, contact, returns and purchases are not implemented.

QUICK LOOKUPS

#OOO1

#OOO2

#OOO3

RECEIPT & DISPATCH LEDGER REFERENCE

#OOO1

Print / Save PDFBack to Search

ORDER #OOO1 Details

CUSTOMER DETAILS

Sample Customer A

Customer ID: OOO1

Local sample customer record

SHIPPING DESTINATION

Sample delivery address A sample state

Status: Shipped

FULFILLMENT PROGRESS: RECORDED EVENTS

STEP Complete01Order placed2026-10-01

STEP Complete02Repacked2026-10-02

STEP Pending03DeliveredPending

ITEMS IN THIS ORDER2 product lines3 units

Apple iPhone 16

Product #1

Unit price \$799.00

Qty: 1\$799.00View Details

Logitech MX Master 3S Bluetooth Edition Mouse

Product #18

Unit price \$95.00

Qty: 2\$190.00View Details

The Archive & Order Ledger

A multi-table, connected sample

[Test Search](#)
[Voice Search](#)
[Image Search](#)
[Order Search](#)

[Saved Products](#)
[AMT](#)

Retrieved order #O001 Shipped

SELECTED ORDER

#001

LEDGER RECORD

Shipped

ORDER #001

CUSTOMER

ORDER DATE

Sample Customer

2026-10-01

A

ID: C001

Recorded events completed

2/5

ORDER TOTAL

\$99.00

MANIFEST

2 product lines

3 units

Print Order Summary

ORDER DISCOVERY NOTE

These three sample records are read from the Python order repository. History, saved invoices, contact, returns and purchases are not implemented.

QUICK LOOKUPS

#0001

#0002

#0003

RECENT LOOKUPS

#0001: Sample Customer A - Shipped

RECEIPT & DISPATCH LEDGER - REFERENCE

#0001

Print / Save PDF

Back to Search

Order #001 Details

CUSTOMER DETAILS

Sample Customer A

Customer ID: C001

Local sample customer record

SHIPPING DESTINATION

Sample delivery address A (sample)

Status:

Status: Shipped

FULFILLMENT PROGRESS - RECORDED EVENTS

STEP	Complete	STEP	Complete	STEP	Pending
01	02	02	02	01	
Order placed	Registered	Invoiced		Pending	
2026-10-01	2026-10-02				

ITEMS IN THIS ORDER

2 product lines: 3 units

Apple iPhone 16

Product #1

Qty: 1

\$799.00

Unit price: \$799.00

View Details

Logitech MX Master 3S Bluetooth Edition Mouse

Product #14

Qty: 2

\$99.00

Unit price: \$99.00

View Details

Subtotal

\$799.00

Shipping

\$0.00

Tax

\$0.00

Total Amount

\$99.00

Payment: Simulated payment — no transaction

Contact Support

Reorder

Update Status

Read-only sample ledger. Use your browser's print dialog to print or save a PDF.